

CS3.301: Operating Systems and Networks

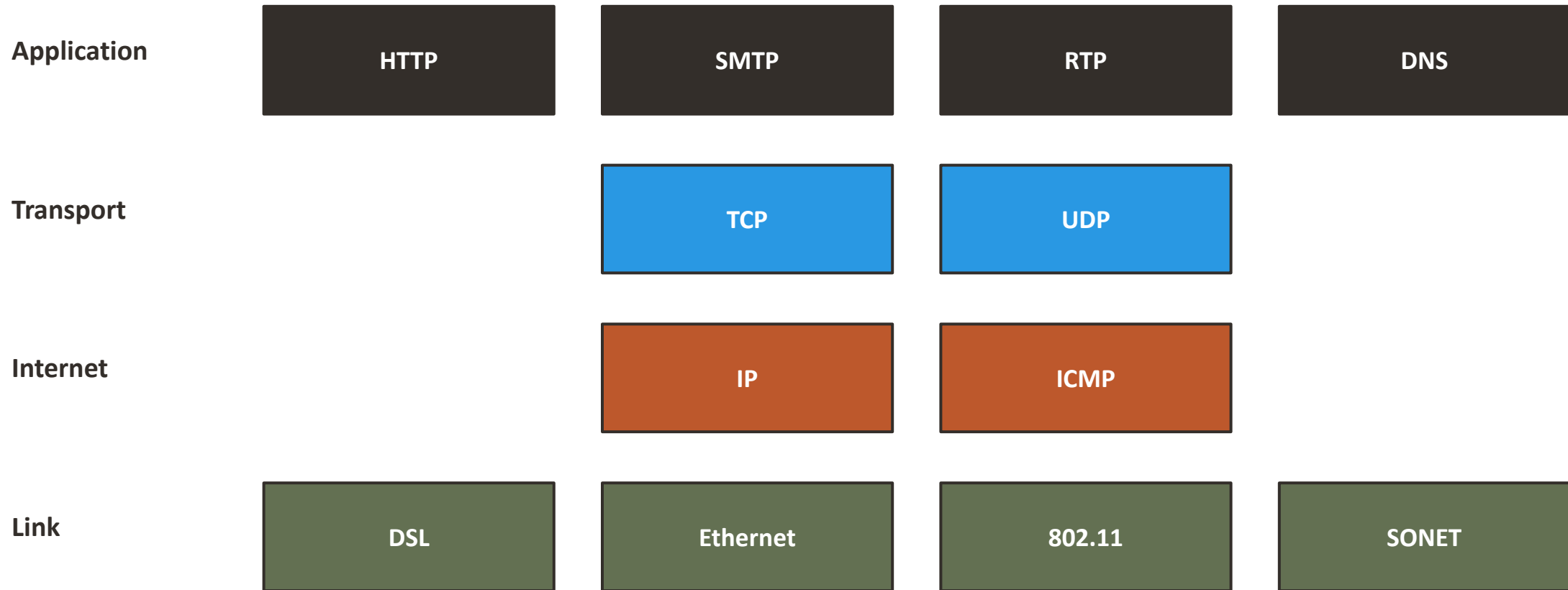
Lecture 9 - Transport Layer (How It Works!)

INSTRUCTOR: DEEPAK GANGADHARAN

Acknowledgement

- The material in this lecture has been gathered, adapted, and generated from multiple sources, combined with the instructor's own experience and knowledge.
- Sources:
 - Computer Networks, 6th Edition — Tanenbaum, Feamster, and Wetherall
 - Computer Networking: A Top-Down Approach — Kurose and Ross
 - Computer Networking Essentials — YouTube channel
 - Other online sources, duly cited where used

Network Protocol Stack (Recap)



Onto the Transport Layer

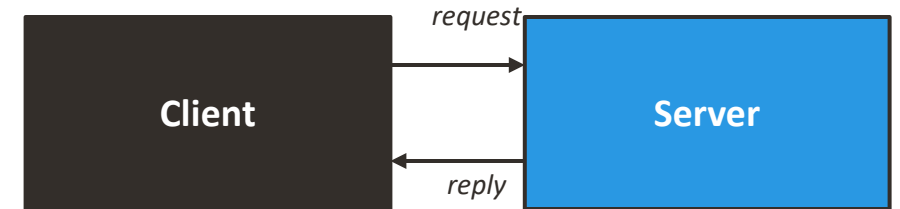
Service-to-Service Delivery

- Multiplexing and Demultiplexing
- Addressing scheme: Ports
- Two strategies/protocols that allow this:
 - Transmission Control Protocol (TCP) — favours reliability
 - User Datagram Protocol (UDP) — favours efficiency

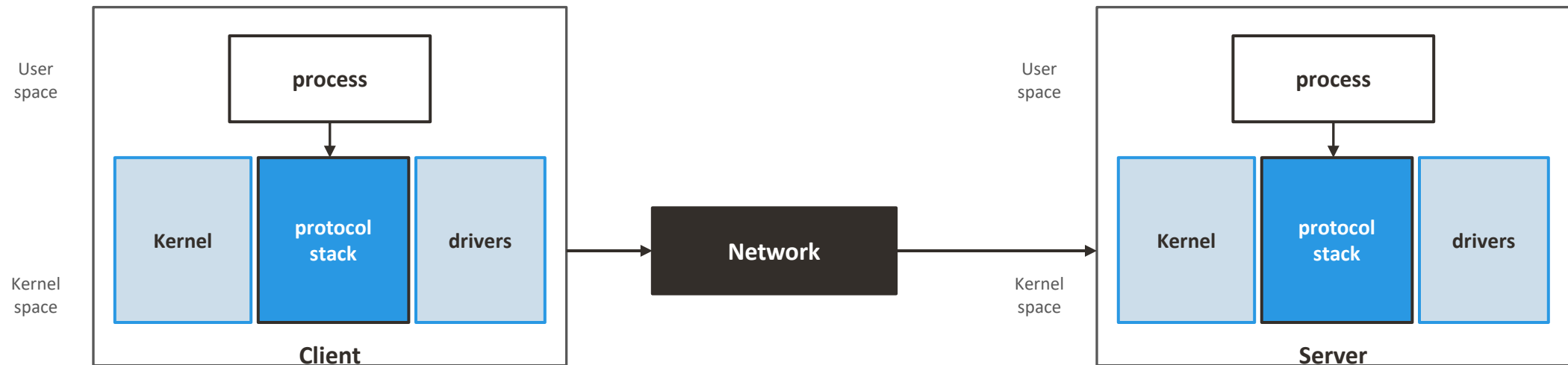
Let's Go Back

Two processes want to communicate

- Client sends a request to the server.
- The server provides a reply based on the request made by the client.
- E.g., file transfer to get files.
- E.g., web browsing — send a URL and get a page back.
- How do we transport data from process to process?



The Role of the Operating System



- The software component in the OS that supports network calls is called the protocol stack.
- It provides service primitives — which are nothing but system calls. Some kind of API?

A Question to Consider

Any idea what functionalities should be made available to let a process use the network?

Hint: think of the process API

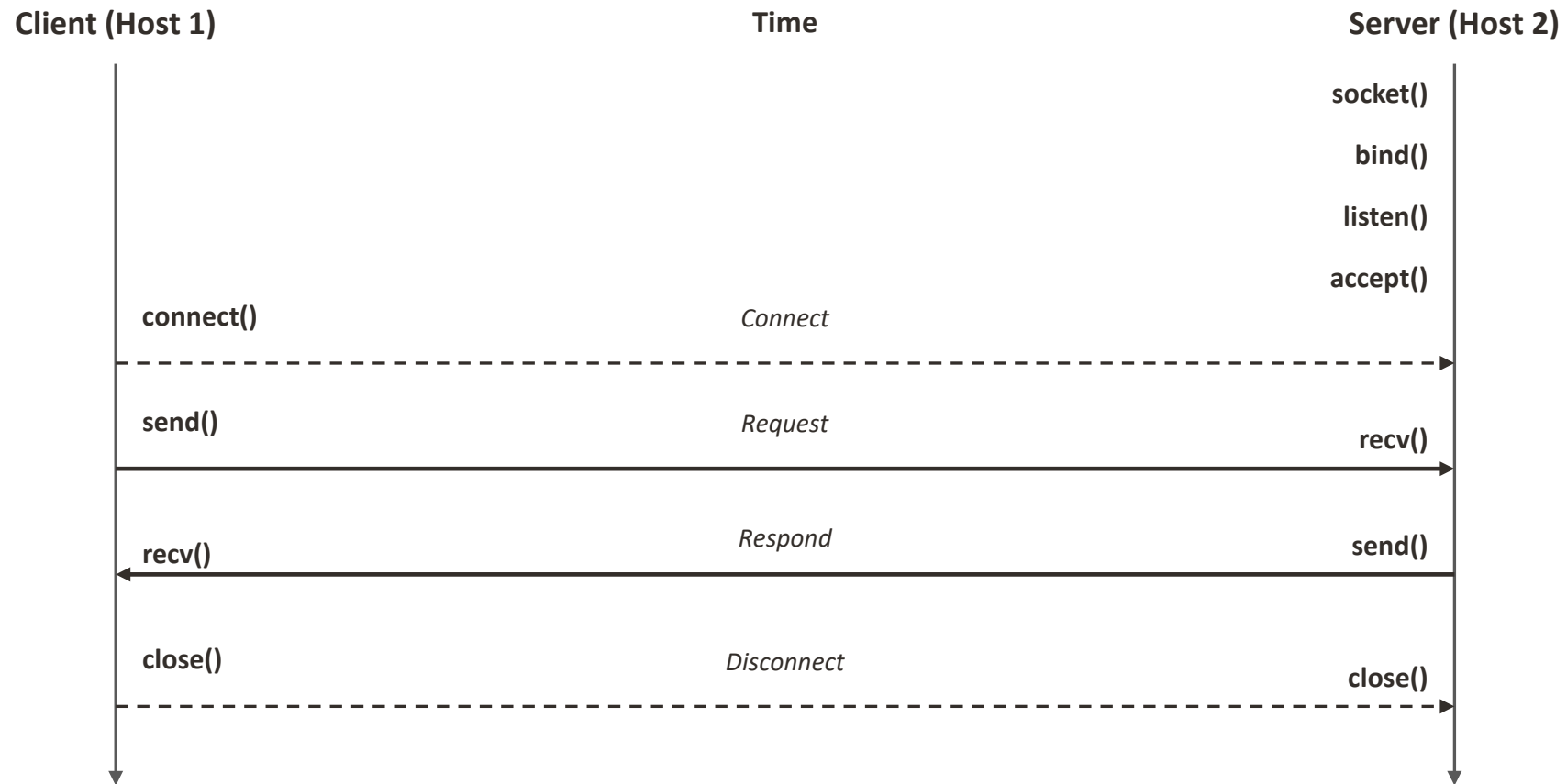
The Socket API

- A simple abstraction for using the network.
- The network service API used to write essentially all network applications.
- Part of every OS and every language [Berkeley Unix, 1983].
 - Allows user-space applications to interact with the networking subsystem.
- Two services:
 - Streams: reliable (connection-oriented).
 - Datagram: unreliable (connectionless).
- Allows applications to attach to the network at different ports.

The Socket API — Core Functions

Function	Description
socket()	Creates a new socket of a certain type (TCP or UDP) and returns a file descriptor
bind()	Associates the socket with a specific IP address and port
listen()	For server sockets — allows the socket to listen for incoming connections
accept()	For server sockets — waits for a client to connect, then returns a new file descriptor
connect()	For client sockets — initiates a connection to a server
send() / recv()	Transmit data or receive data
close()	Terminates the connection

Using Sockets



More About Ports

- An application process is identified by the tuple (IP address, Protocol, Port).
 - Ports are 16-bit integers representing "mailboxes" that a process leases.
- Servers are often bound to "well-known ports."
- Clients are assigned ephemeral ports.
 - Chosen by the OS temporarily.

Some Well-Known Ports

Port	Protocol	Use
20, 21	FTP	File Transfer
22	SSH	Remote login
25	SMTP	Email
53	DNS	Domain name resolution
80	HTTP	World Wide Web
443	HTTPS	Secured web
554	RTSP	Media player control

An Opportunity for a Context Switch?

- The calls involved in establishing a socket connection are blocking calls:
 - `connect()`, `accept()`, `recv()`
- Once the call is made, the OS halts the program to wait for some response.
- These are essentially system calls.
- A trap instruction is executed, and there is an opportunity for a context switch.

Let Us Take a Step Back

Types of Links

- Full Duplex

- Bidirectional — both sides can send at the same time.

- Half-Duplex

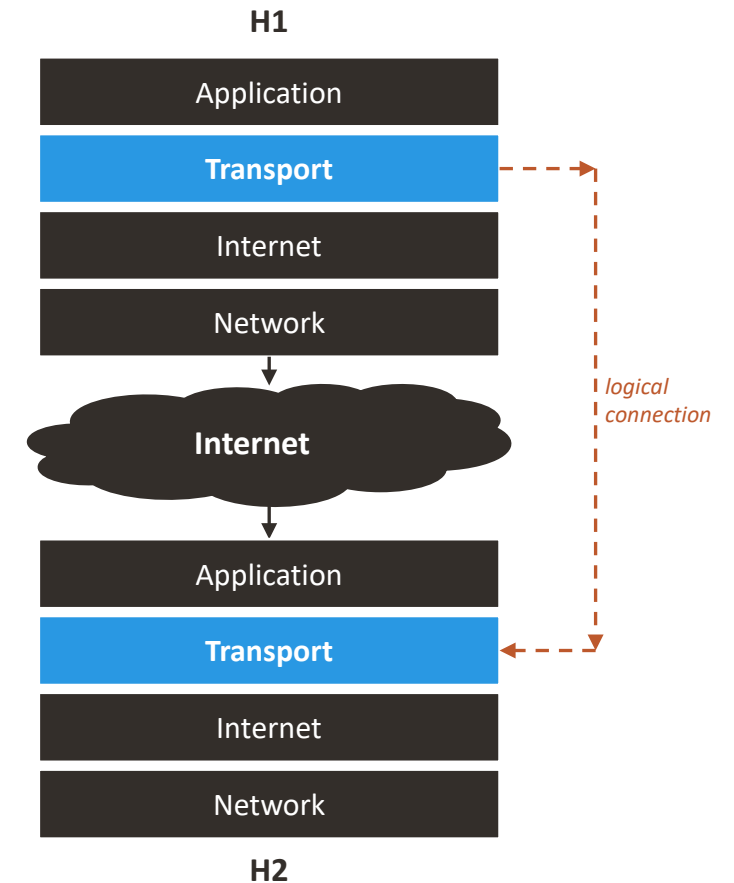
- Bidirectional — but only one direction at a time (e.g., walkie-talkies).

- Simplex

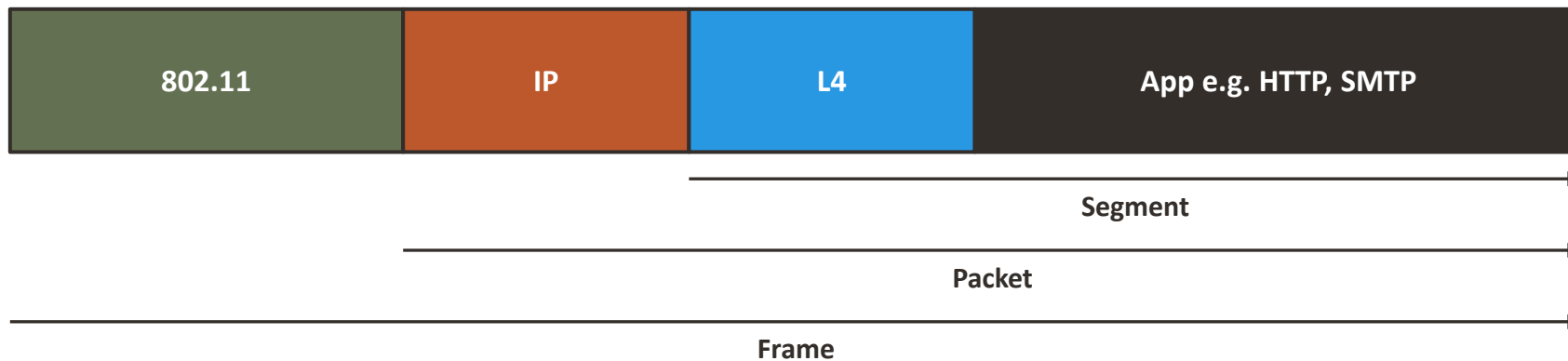
- Unidirectional only (e.g., a keyboard sending input, or a TV broadcast).

Transport Services and Protocols

- Provides logical communication between application processes running on different hosts.
- Transport protocol actions in the end systems:
 - Sender: breaks application messages into segments, passes to the network layer.
 - Receiver: reassembles segments into messages, passes to the application layer.
- Protocols: TCP, UDP



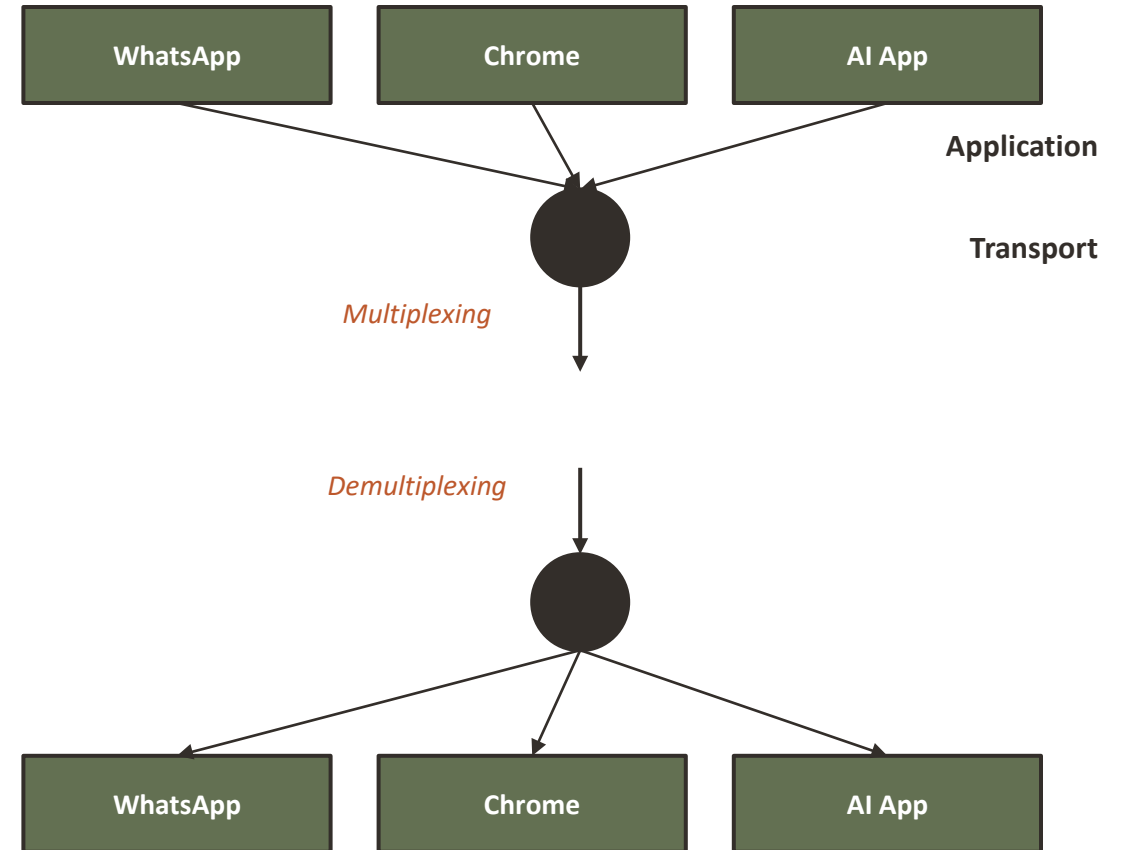
Quick Recap



- Segments carry data across the network.
- Segments are carried within packets, which are carried within frames.
- Each layer adds a header. (Anything above L4 gets bundled and treated as the L4 payload.)

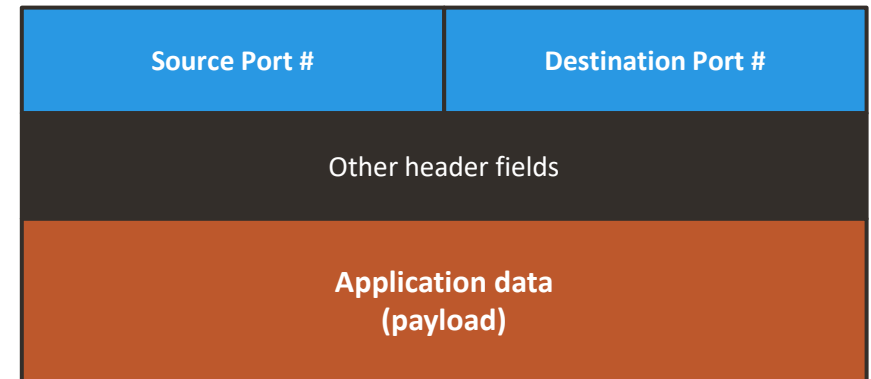
Multiplexing and Demultiplexing

- Multiplexing (as sender): handle data from multiple sockets, add a transport header.
- Demultiplexing (as receiver): use header info to deliver received segments to the correct socket.



Working of Demultiplexing

- Host receives IP datagrams.
 - Each datagram has a source IP address and a destination IP address.
 - Each datagram carries one transport-layer segment.
 - Each segment has a source port number and a destination port number.
- IP addresses and ports together are used to direct a segment to the appropriate socket.



TCP/UDP Segment Format

Connection-Oriented vs. Connectionless

Demultiplexing Scenarios

■ Connection-Oriented (TCP)

- TCP socket identified by a 4-tuple.
 - Source IP, destination IP, source port, destination port.
- Receiver uses all 4 values to direct a segment to the right socket.
- A server may support many TCP sockets — each socket has its own client.

■ Connectionless (UDP)

- UDP socket identified by a 2-tuple.
 - Destination IP and destination port.
- Receiver uses just the port to redirect to the corresponding socket.
- UDP segments with the same destination port but different source IP/port are redirected to the same socket.

TCP vs UDP

TCP	UDP
Connection Oriented	Not Connection Oriented
Reliability (order maintained, retransmission)	No reliability at L4
Higher overhead — reliability, error checking, etc.	Low overhead
Flow control (based on network conditions)	No implicit flow control
Error detection — retransmits erroneous packets	Some error checking — erroneous packets discarded without notification
Congestion control	No congestion control
Use cases: HTTP/HTTPS, file transfer, mail	Use cases: streaming data, VoIP, DNS queries, ...

Connection-Oriented and Reliability

■ Connection-Oriented

- In TCP, a connection is established first, before any data is transmitted.
- In UDP, there is no notion of a connection starting or ending (relies on timeout instead).

■ Reliability

- TCP confirms data delivery via acknowledgments.
 - Order is preserved / maintained.
 - Errors can be detected and handled (awareness) — TCP can recover from them.
- In UDP there is no confirmation — the sender simply trusts that someone is there to receive the data ("fire and forget").
 - No error awareness at L4 — the protocol itself does not handle it.

Flow Control and Overhead

■ Flow Control

- TCP can adjust its transmission rate to use the maximum available bandwidth.
- It checks how much the receiver can actually receive, and adjusts accordingly.

■ Overhead

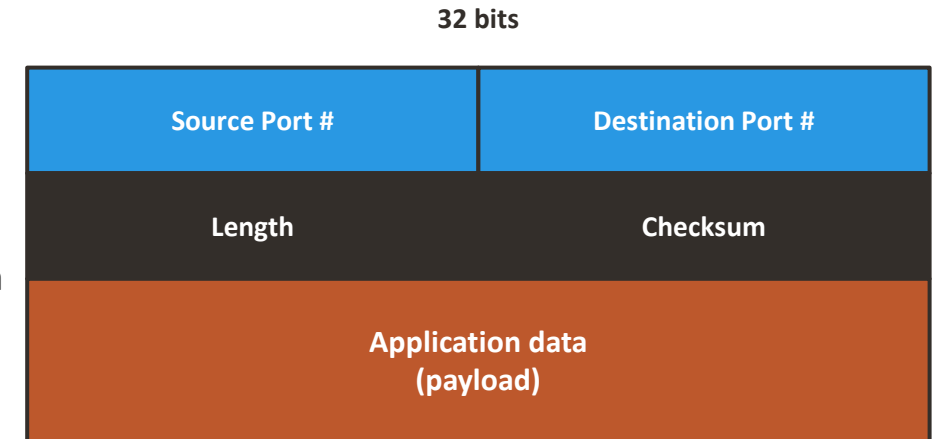
- TCP adds a larger header to the data — roughly 20 bytes or even more.
- TCP has more built-in features that simply don't exist in UDP.
- In UDP, the header length is only about 8 bytes.



L4 header + payload

UDP Segment Header

- Length: the size in bytes of the UDP segment, including the header.
- Checksum: for error detection.
 - A 16-bit value representing the sum of the UDP header, payload, and a pseudo-header from the IP layer.
 - Supports error detection.
 - Makes use of one's-complement arithmetic to compute the sum.



UDP Segment Format

Checksum Process

■ Sender

- All contents of the header, including IP addresses, are treated as a sequence of 16-bit integers.
- Checksum = addition (one's complement) of the segment's content.

Example

1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0

1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1

Add it back 1 1 0 1 1 1 0 1 1 1 0 1 1 1 0 1 1

Sum 1 0 1 1 1 0 1 1 1 0 1 1 1 1 0 0

Checksum 0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 1

■ Receiver

- Computes the checksum of the received content.
- Checks if the received checksum and the header's checksum are equal — if so, no error.
- Otherwise, an error is detected.

TCP: A Closer Look

**TCP is the most used protocol on the Internet. How does it actually work?
What would you need to build in order to provide the features TCP promises?**

Thank You!